

CVM++

Setup, structure, and language overview

5-page presentation · github.com/Dragon9-9/cvmpp

Satish Kumar Munda · github.com/Dragon9-9/cvmpp · C++17 Personal Project

What is CVM++?

A **scripting language** + full **toolchain** in C++17 — one binary `cvmpp`, no LLVM/Python.

.cvm → Lexer → Parser → Compiler → Stack VM → Output

tokens · AST · bytecode · execution

Piece	Role
Language	.cvm — variables, fn, if/while, print, input
Lexer / Parser / Compiler	Source to bytecode bytes
VM	Stack execution + safety checks
CLI / REPL	Files, debug, interactive mode

Build and run

Clone and build

```
git clone https://github.com/Dragon9-9/cvmpp.git
cd cvmpp && make
make verify          # 11/11 tests
```

Run

```
./build/cvmpp examples/hello.cvm
./build/cvmpp -d examples/functions.cvm
./build/cvmpp          # REPL
```

Flags

Flag	Effect
-d	Tokens + AST + bytecode
-q	Quiet output
-c	Compile only

Exit: 0 ok · 1 compile · 2 runtime

Repository layout

```
include/cvm++/  headers
src/            lexer, parser,
               compiler, vm, main
examples/*.cvm  test scripts
scripts/        verify.sh
Makefile        make / make verify
```

Built in 5 phases

1. **Phase 1** — lexer, tokens, diagnostics, UI
2. **Phase 2** — parser, AST
3. **Phase 3** — bytecode, compiler
4. **Phase 4** — stack VM, values
5. **Phase 5** — REPL, functions, CI

Functions at start of bytecode; offset-0 JUMP skips to **main**.

CVM++ language

```
fn factorial(n) {
  if (n <= 1) { return 1; }
  return n * factorial(n - 1);
}
print factorial(5); // 120
```

Feature	Syntax
Types	int, bool
Vars	let x = ...; x = ...;
Functions	fn f(a) { } f(1,2)
Control	if/else while
Ops	+ - * / == != < > <= >=
I/O	print input

REPL commands

help quit debug run file.cvm disasm / disasm
file.cvm

Not supported

strings, floats, arrays, for, modules

Examples and verification

Script	Shows
hello.cvm	let, if, print
functions.cvm	recursive fn → 120
factorial.cvm	while loop
comparisons.cvm	!= <= >=
div_by_zero.cvm	runtime error exit 2

make verify — 11/11 PASSED hello · arithmetic · booleans · if_else · factorial · multiline_demo · assignment · functions · comparisons · input_demo · div_by_zero

Project status

Component	Status	Verified
Lexer	Tokens, overflow, comments	✓
Parser	AST, if/while/fn	✓
Compiler	Bytecode, jumps, calls	✓
Stack VM	Arithmetic, frames, failsafes	✓
CLI / REPL	Files, -d, multiline	✓
CI	GitHub Actions on push	✓

Personal project — full interpreter pipeline in C++17.
Inspired by **Crafting Interpreters** (Robert Nystrom).
No external interpreter libraries.

Implementation milestones

Phase	Deliverable	Test
1 Foundation	Lexer + diagnostics + UI	Token table prints
2 Parser	AST + recursive descent	AST tree in -d
3 Compiler	Bytecode + disasm	:disasm works
4 VM	Stack execution	print 1+2 → 3
5 Product	REPL, fn, CI, examples	make verify 11/11

Key terms

Term	Meaning
Token	Lexer output unit (keyword, number, symbol)
AST	Syntax tree after parser
Bytecode	vector<uint8_t> VM instructions
Opcode	One-byte command: Add, Call, Jump...
Stack	Expression values during execution
Call frame	Return address + locals in a function
VmSession	REPL globals that persist between lines
Diagnostic	Error with phase, line, hint